AutoEncoder y clasificador convolucional sobre Fashion-MNIST

Gonzalo Angaut
Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Medina Allende S/N, Coŕdoba, Argentina
(Dated: February 6, 2024)

En este estudio, se evalúa el rendimiento de una red neuronal autoencoder convolucional, seguido por la utilización del encoder para desarrollar un clasificador destinado al conjunto de datos Fashion-MNIST. Se analiza el desempeño de ambos componentes a través de métricas clave, como el error y la precisión, y se presenta una evaluación visual mediante ejemplos específicos. Finalmente, la efectividad del modelo se evalúa mediante el análisis de una matriz de confusión, proporcionando una conclusión sobre su capacidad para clasificar con precisión las prendas de vestir en el conjunto de datos.

I. INTRODUCCIÓN

El mundo de la inteligencia artificial es tan emocionante como expansivo. Entre las técnicas más poderosas se encuentran los autoencoders, instrumentos del aprendizaje no supervisado que buscan capturar la esencia de los datos al comprimir y reconstruir la información de entrada. Por otro lado, los clasificadores convolucionales destacan por su eficacia en el procesamiento de datos estructurados, especialmente en el ámbito visual, como en imágenes.

Este informe se sumerge en el uso de estas herramientas en Fashion-MNIST, una base de datos de moda. Vamos a ver cómo los autoencoders pueden comprimir y reconstruir datos, y cómo los clasificadores convolucionales destacan en la clasificación visual.

II. ENFOQUE TEÓRICO

Las redes neuronales han tenido un gran crecimiento durante las últimas décadas. Entre ellas, el perceptrón simple, propuesto por Rosenblatt en 1957, se destaca como la forma más elemental de una red neuronal, lo que lo convierte en un recurso didáctico invaluable. Sin embargo, las limitaciones inherentes de los perceptrones simples impulsaron la evolución hacia las redes multicapa. Es importante destacar que incluso una red con una sola capa oculta puede representar cualquier función booleana, subrayando así la versatilidad y el potencial de las redes neuronales más complejas. [1]

En el ámbito de aprendizaje no supervisado, los autoencoders desempeñan un papel crucial. Estas redes buscan comprimir la información de entrada en una representación de menor dimensión y luego reconstruir la entrada original a partir de esta representación, logrando obtener representaciones más eficientes y comprimidas de los datos. La arquitectura de un autoencoder se compone típicamente de una entrada, un codificador que reduce la dimensión, una representación interna, un decodificador que reconstruye la entrada, y una salida, todo respaldado por una función de pérdida que cuantifica la calidad de la reconstrucción (ver Figura 1). [2]

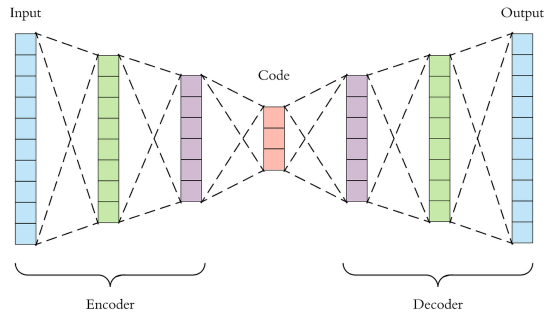


FIG. 1: Esquema cualitativo de un autoencoder.

En contraste, las redes convolucionales, que operan en una topología de tipo grilla y utilizan convoluciones en lugar de la multiplicación de matrices, han emergido como una herramienta extraordinariamente efectiva. Este tipo de redes ha demostrado un éxito significativo en diversas aplicaciones prácticas, especialmente en el procesamiento de imágenes.[2]

La clasificación en redes neuronales, esencial en aprendizaje automático, implica asignar etiquetas o categorías a datos de entrada. Las redes neuronales sobresalen en esta tarea al aprender representaciones complejas y no lineales de los datos, facilitando la clasificación efectiva de patrones en diversas aplicaciones.

En la implementación de la red neuronal durante este trabajo, se emplearon diversas herramientas para optimizar el rendimiento del modelo. Por ejemplo el uso del dropout, una técnica que apaga aleatoriamente algunas neuronas durante el entrenamiento (ver Figura 2), se reveló fundamental para mitigar el sobreajuste y mejorar la generalización del modelo. Además, se exploraron distintas funciones de activación cuya función principal es agregar componentes no lineales a la red. En particular, se decidió utilizar *ReLU*, la función de activación más famosa de la actualidad y útil sobretodo en capas ocultas para aprender patrones complejos teniendo como particularidad que no está acotada, y *Sigmoid* que acota los resultados entre 0 y 1 por lo que suele usarse en la capa de salida en modelos de clasificación binaria cuando

se necesita interpretar la salida como probabilidades o en casos de imágenes en escala de grises.

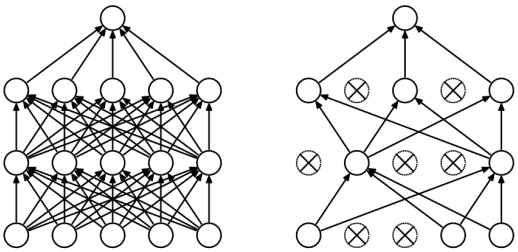


FIG. 2: Red neuronal antes y después de aplicar dropout.

Para respaldar este estudio, se empleó la base de datos Fashion-MNIST, que comprende un conjunto de entrenamiento de 60,000 ejemplos y uno de prueba de 10,000 ejemplos. Cada instancia es una imagen en escala de grises de 28x28 píxeles, asociada con una de las diez etiquetas de clases disponibles.[3]

III. PROCEDIMIENTO

El presente trabajo estuvo dividido en dos etapas: primero, en la definición y entrenamiento de un autoencoder y, luego, de un clasificador.

Por definición, el autoencoder consiste de dos partes: un encoder (codificador) y un decoder (decodificador). El propósito de la utilización del autoencoder en el trabajo es el de entrenar al encoder para luego reutilizarlo en la parte del clasificador.

En la Figura 3 puede verse un esquema del encoder. Éste es alimentado por un input (en este caso, por una imagen de la base de datos Fashion-MNIST) que pasa por una primera capa convolucional 2d compuesta por una capa *Conv2d* que tiene como entrada 1 canal de 28x28 píxeles y como salida 16 canales de 26x26 utilizando una función de activación de tipo *ReLU* y *dropout* con $p = 0.2$, y una capa *Maxpool* que reduce las dimensiones de los 16 canales a la mitad. Luego, sigue otra capa convolucional 2d compuesta, nuevamente, por una capa *Conv2d* pero que esta vez tiene como entrada 16 canales de 13x13 y como salida 32 canales de 11x11 cuya dimensión se reduce a la mitad gracias a una capa *Maxpool* posterior. Ambas capas *Conv2d* tienen un kernel de dimensiones $(3, 3)$, mientras que las *MaxPool* tienen uno de $(2, 2)$. Por último, el encoder tiene una capa lineal. Ésta tiene una entrada de dimensiones $(32, 5, 5)$ y la colapsa en un vector unidimensional de 32x5x5 gracias a una capa *Flatten*. Finalmente, una capa *Linear* con misma función de activación de antes e igual *dropout* nos lleva a una capa de 64 neuronas, que es la imagen codificada.

Por otro lado, en la Figura 4 puede verse el decoder

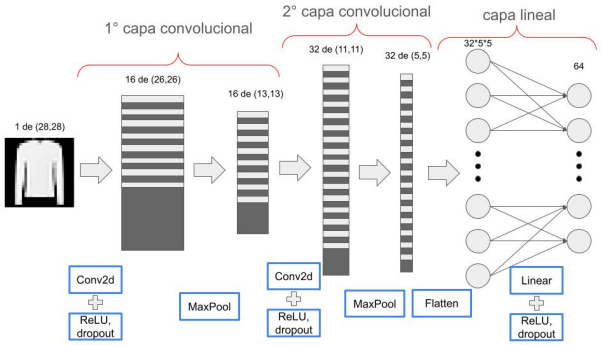


FIG. 3: Esquema del Encoder.

definido. Empieza con una capa lineal que consiste en una *Linear* con *ReLU* y *dropout* que mapea la capa de 64 neuronas (la imagen codificada) con una de 32x5x5 y luego un *UnFlatten* con salida de dimensiones $(32, 5, 5)$. La sigue la primera capa convolucional transpuesta (a la segunda capa convolucional del encoder) consistente de una *ConvTranspose2d* con *ReLU* y *dropout* con salida de dimensiones $(16, 13, 13)$. Por último, hay una segunda capa convolucional transpuesta (a la primera del encoder) compuesta de una *ConvTranspose2d* y cambiando la función de activación a una una *Sigmoid*. Esta capa tiene como salida a una imagen de 28x28 píxeles (tal como la ingresada al encoder). La primera capa convolucional transpuesta tiene un kernel de dimensiones $(4, 4)$, mientras que la segunda de $(3, 3)$ y ambas tienen un stride de $(2, 2)$ y un out padding de $(1, 1)$.

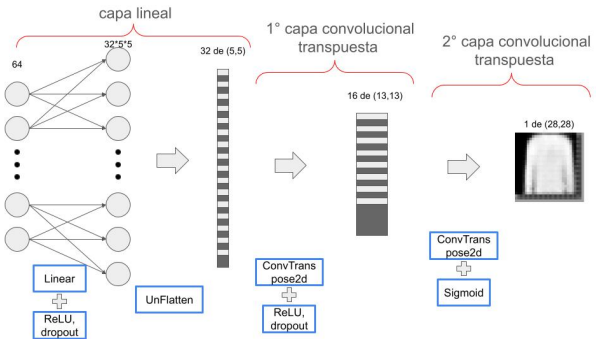


FIG. 4: Esquema del Decoder.

Una vez definido el autoencoder, se utilizan los 60.000 ejemplos del conjunto de entrenamiento de la base de datos para entrenar a la red. Simultaneamente al entrenamiento, se realiza una validación sobre las 10.000 imagenes del conjunto de prueba. En particular, se utilizaron minibatchs de tamaño 1.000, se usó ADAM como optimizador, la función Error Cuadrático Medio como función de pérdida y se entrenó a la red por 30 épocas.

Al finalizar el entrenamiento del autoencoder, se definió la red de clasificación. Un esquema de ésta puede verse en la Figura 5. Se alimenta a la red con una im-

agen de la base de datos y se la hace pasar por el encoder pre-entrenado. A la última capa del encoder (con 64 unidades) se la mapeó, de forma lineal y con función de activación *ReLU*, con la capa de salida que tiene 10 neuronas, una relacionada con cada clase de prenda.

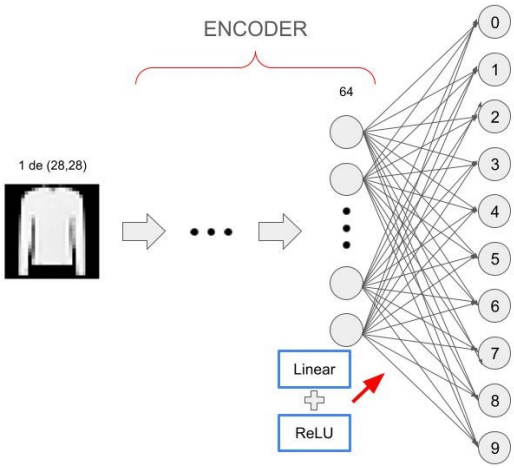


FIG. 5: Esquema del clasificador.

Para el clasificador se utilizó ADAM como optimizador, la función Cross Enropy Loss como función de pérdida y fue entrenado con minibatchs de tamaño 100 y 30 épocas.

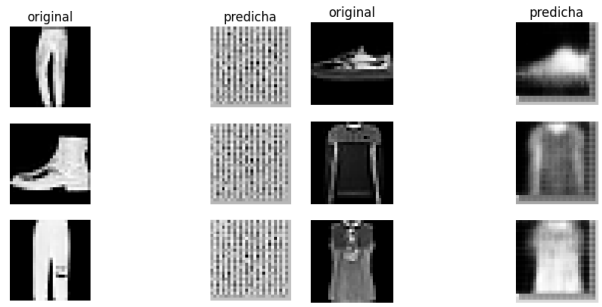
IV. RESULTADOS

En la Figura 6 se analiza el comportamiento del autoencoder. Pueden verse 3 ejemplos de imágenes relacionadas al modelo antes de entrenar y otros 3 ejemplos despues del entrenamiento. Asimismo, están graficadas las curvas de error para los datos de entrenamiento y validación en función de las épocas. Puede verse cómo, luego de entrenar la red, las imágenes son reconstruídas conservando la estructura original y, además, el error disminuye casi de igual forma para los datos de entrenamiento y de validación estancándose cerca de 0.04.

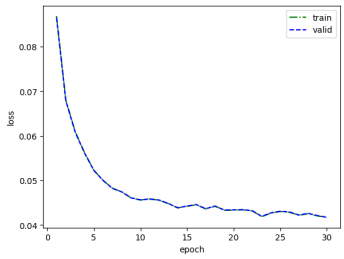
Lo mismo puede verse en la Figura 7 para el clasificador, donde además se añadió un gráfico de la precisión. En ambos gráficos se aprecia un cambio abrupto entre las épocas 5 y 10, mientras que luego el cambio se hace de forma más gradual. Se puede ver cómo la precisión es muy alta ya que luego de las 30 épocas está por encima del 90%.

En la Figura 8 pueden verse algunos ejemplos aleatorios de lo que predice nuestro modelo para algunas imágenes del conjunto de validación la base de datos. En los ejemplos donde la predicción es correcta se puede ver cómo el porcentaje es muy alto, mientras no sucede lo mismo donde la red falla.

Por último, en la Figura 9 se observa una matriz de

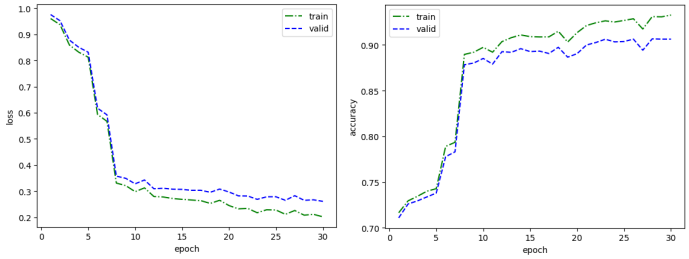


(a) (b)



(c)

FIG. 6: Comportamiento del autoencoder. (a) Imágenes antes de entrenar. (b) Imágenes despues del entrenamiento. (c) Curvas del error de los datos de entrenamiento y validación.



(a) (b)

FIG. 7: Comportamiento del clasificador. (a) Imágenes antes de entrenar. (b) Curvas del error de los datos de entrenamiento y validación. (c) Curvas de la precisión de los datos de entrenamiento y validación.

confusión que muestra qué predicciones hizo el modelo para cada una de las clases. Nótese que en todos los casos la red suele predecir de manera correcta, formando la diagonal en la matriz.

V. CONCLUSIÓN

En resumen, este estudio ha alcanzado el desarrollo de una red neuronal altamente precisa en la tarea de clasificación de imágenes de la base de datos Fashion-MNIST.

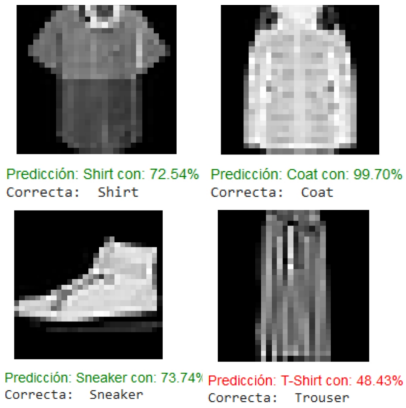


FIG. 8: Algunos ejemplos de predicciones.

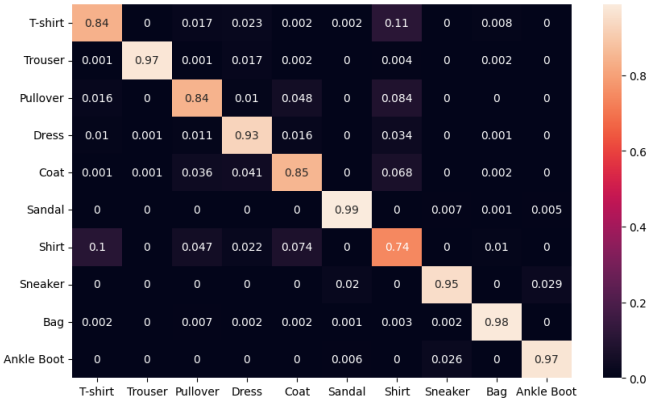


FIG. 9: Matriz de confusión.

La implementación de un autoencoder convolucional ha demostrado ser crucial, logrando reconstruir las imágenes de entrada mientras conserva la estructura original. Utilizando el encoder, se construyó un clasificador con una impresionante precisión cercana al 90% (ver Figura 7). La presentación de ejemplos visuales respalda de manera efectiva el rendimiento sobresaliente de la red, y la matriz de confusión revela cómo el clasificador logra aciertos consistentes para cada categoría de prendas de vestir (Figura 9).

Estos resultados consolidan la eficacia de la arquitectura propuesta y destacan su aplicabilidad en la clasificación precisa de conjuntos de datos complejos como Fashion-MNIST. Este trabajo no solo aporta a la comprensión teórica, sino que también ofrece una contribución práctica al campo del aprendizaje profundo y la visión por computadora.

[1] Hertz, J. et al. *Introduction to the neural computation.* Santa Fe Institue, Studies in the sciences of complexity.
[2] Bengio, Y. et al. *Deep Learning* (2015).
[3] https://github.com/zalandoresearch/fashion-mnist